Document number: P3165R0
Audience: SG21

Ville Voutilainen
2024-02-26

Contracts on virtual functions for the Contracts MVP

Abstract

This paper proposes adding contracts on virtual functions, to the contracts MVP. The overall rationale is that we shouldn't postpone something as important as that post-MVP, as that would violate various completeness design principles of C++, including supporting all the major techniques and styles of C++. Virtual functions remain a facility of fundamental importance in C++, and not supporting contract checks on virtual functions would be untoward enough to be called a travesty.

In a nutshell, the proposed approach is to not inherit base class virtuals' contracts into the definition of an override, in any way. If the function is called bypassing the virtual call mechanism, i.e. with explicit qualification, only the function's own contracts are evaluated. If a function is called via the virtual call mechanism, the contracts of the base function in the type used for the call are evaluated, in addition to the function's own contracts.

1. Design goals

This proposal has the following design goals, ones that some could call design principles:

- the approach must work with existing ABIs, so the proposal should have no ABI impact
- the "canonical" substitutable designs must be relatively easy to express, but do not need to be enforced, because..
- ..the design must allow less-canonical designs, for instance where an overrider has a stricter contract than its base
- the approach must not require elaborate proof of subsumption of contracts in a hierarchy
- the approach must seek to maintain as much of the possibility to do caller-side checking as possible, but no more

The rationale for these is straightforward:

- It would be undesirable to the point of being unacceptable if adding, removing, or changing contracts in a class hierarchy causes programs to no longer link. Don't get me wrong, it's not hard to imagine where something like that could be beneficial, but it's also very easy to not just imagine but actually find real-life scenarios where such breakage would be completely unacceptable, and prevents ever adding contracts to any virtually-dispatched API.
- There are very reasonable designs where the contract of an override is not exactly the same as that of the base, and very plausible cases of that are such where the derived class requires additional (pre)conditions to be established, conditions that cannot necessarily be reasonably expressed as contracts of the base function, if at all.
- We must, however, support common designs, and designs where overriders have exactly the same contract as their base are common, even when such overrides are called directly, bypassing the virtual call mechanism.
- It should hopefully be very easy to understand why there's a design goal of not requiring subsumption proofs. We are talking about arbitrarily complex expressions with arbitrary runtime behavior, so computing subsumption proofs for those would, if doable at all, require much more than we have in the MVP, and much more than we can ostensibly expect having, and restrictions that we don't want to have.
- The possiblity to do caller-side checking is attractive, we shouldn't just drop that possibility if we can avoid doing so. There are, of course, some limitations to how much of that can be done for a hierarchy.

2. The proposed semantics

As hinted in the abstract, the idea is that a virtual call checks the contracts of the overrider it dispatches to, and the base function contract of the type used for the call as well, whereas a direct qualified call checks only the contracts of the function it's calling.

The contract on an overrider is completely independent from the contract of the base function. That is, it need not be the same, it need not have any sort of subsumption relationship with the contract of the base function, it can be wider, it can be narrower. And there is no inheritance of the base contract into the contract of the overrider.

Preconditions and postconditions are handled in a similar fashion; the preconditions and postconditions of the "entry point" used for the call are checked, and preconditions of the final overrider actually called are checked after the preconditions of the base function, and the postconditions of the final overrider are checked before the postconditions of the base function.

The intent is to check just the contract-pair of an "entry point" and final overrider, and there is no checking of a chain of contracts in the hierarchy of intermediate classes that are bases of the class of the final overrider and inherit the class of the "entry point". The reason for

this is three-fold:

- the field isn't green, this mechanism aims to (following pun intended) slot in to the existing virtual call mechanism, and an important goal is that there is no ABI impact, no additional vtbl slots needed, no ping-pong between the "entry point" view and the final overrider view.
- I'm not sure whether there are other strong reasons. From a certain perspective, a virtual call is conceptually an interface-implementation pair, and intermediate layers don't play into it. Trying to get them to play seems to suggest elaborate mechanisms that don't necessarily manage to meet the "no ABI impact required" goal. A call site doesn't know how deep a hierarchy it's calling in to, the final overrider site doesn't know which layer in a hierachy was the "entry point". There are existing cases where adding or removing layers is neither an API nor an ABI break, it's unclear how much of that could be retained with a more elaborate chaining mechanism.

Let's go straight into an illustrating example:

```
 // Example 2.1:
struct Vehicle
{
      virtual void drive(int speed) pre(speed_within_limit(speed)); // #2.1.1
};

struct MotorVehicle : Vehicle
{
      bool engineRunning = false;
      void drive(int speed) pre(engineRunning) override; // #2.1.2
};

void use1(Vehicle* veh)
{
      veh->drive(80); // #2.1.3
}

void use2()
{
      MotorVehicle mv;
      use1(&mv);
      mv.drive(400); // #2.1.4
}
```

So, the intent is that the use at #2.1.3 checks both the contract at #2.1.1 and the contract at #2.1.2. In other words, it's a virtual call, it dispatches to #2.1.2, but since it was called via the virtual call mechanism, it also checks the base contract.

The use at #2.1.4 is still conceptually a call using the virtual dispatch mechanism, but it checks only the contract at #2.1.2. It's not a direct call of that function, but since the type used for the call is the same as the type the overrider is a member of, or in other words, the static and the dynamic type are the same, there's effectively just one contract to check.

Okay then. Let's add more layers:

```
 // Example 2.2:
struct Vehicle
{
      virtual void drive(int speed) pre(speed_within_limit(speed)); // #2.2.1
};

struct WheeledVehicle : Vehicle
{
      bool tiresSufficientlyInflated = false;
      void drive(int speed) pre(tiresSufficientlyInflated) override; // #2.2.2
};

struct MotorVehicle : WheeledVehicle
{
      bool engineRunning = false;
      void drive(int speed) pre(engineRunning) override; // #2.2.3
};

void use1(Vehicle* veh)
{
      veh->drive(80); // #2.2.4
}

void use2()
{
      MotorVehicle mv;
      use1(&mv);
      mv.drive(400); // #2.2.5
}
```

The intent is that the use at #2.2.4 checks the contract at #2.2.1 and the contract at #2.2.3. In other words, it's a virtual call, it dispatches to #2.2.3, but since it was called via the virtual call mechanism, it also checks the base contract of the type used for the call.

The use at #2.2.5 is still conceptually a call using the virtual dispatch mechanism, but it checks only the contract at #2.2.3. It's not a direct call of that function, but since the type used for the call is the same as the type the overrider is a member of, or in other words, the static and the dynamic type are the same, there's effectively just one contract to check.

Okay, fine, these are what could be called "sunshine scenarios", nothing too difficult to understand in any of that, although the way the direct calls work may already be debatable to some. But hold that thought, let's rain on this parade for a little bit, with explanations coming later:

```
 // Example 2.3:
struct Vehicle
{
      virtual void drive(int speed) pre(speed_within_limit(speed)); // #2.3.1
};

struct WheeledVehicle : Vehicle
{
      bool tiresSufficientlyInflated = false;
      void drive(int speed) pre(tiresSufficientlyInflated) override; // #2.3.2
};

struct MotorVehicle : WheeledVehicle
{
      bool engineRunning = false;
      void drive(int speed) pre(engineRunning) override; // #2.3.3
};

void use1(WheeledVehicle* veh)
{
      veh->drive(80); // #2.3.4
}

void use2()
{
      MotorVehicle mv;
      use1(&mv);
      mv.drive(400); // #2.3.5
}
```

Observe the difference in the parameter of use1(): it now takes a WheeledVehicle*, not a Vehicle*.

Here, the call at #2.3.4 checks a contract at #2.3.2 and #2.3.3, and not the contract at #2.3.1.

3. How should this be implemented?

The expected implementation strategy is such that for a particular member function, the contracts attached to a function itself are handled as usual, so for definition-side checking, the function definition is amended with a call to an internal function that checks those contracts. And that's it. The implementation is no different, considering definition-side checking, than checking any contract.

For the part of checking the contract of a base function based on the class used for the call, client-side checking is a plausible and likely implementation strategy. It is of course also fully conforming to just treat that part as always having the 'ignore' semantic. In addition, there are plausible ways via which the whole check need not be completely done on the client-side, such as calling a contract-checking function generated when compiling the definition of the base function.

In order to *automatically*, without any effort from the programmer, also check the contract on a base function, an implementation might do it so that it checks those contracts on the call site. But it doesn't have to, because an implementation can decide that those contract checks always have the "ignore" semantic, and that's fine. We'll get to how programmers can make checks more guaranteed later.

4. First look back at our design goals and our proposed semantics

At this point we are really going to look at just the first goal, "must work with existing ABIs, no ABI impact".

As explained, the addition, removal, or modification of a contract in a virtual function hierarchy should not be an ABI break. It would be dreadful if it were, that would completely ruin what virtual functions do, especially the part where you can change a definition without that affecting the interface, without changing or recompiling your callers. The proposed semantics are in concert with this goal, there's either a direct function call in the vtbl, or a type-adjusting thunk (which is an existing thing), or a (possibly type-adjusting) contract-checking thunk. The size and the layout of the vtbl don't change if the contracts change. In other words, there are no special additional vtbl slots, so there's no churn of the amount of such things changing.

5. Non-canonical designs

Here we go slightly out of order from our initial design goal listing:

- ..the design must allow less-canonical designs, for instance where an overrider has a stricter contract than its base

The approach supports this fine. Since the overrider's contracts are checked, and that check doesn't incorporate any of the base functions' checks, the overrider can have a narrower contract, it can have a wider contract, it can do what it pleases.

Some might find such designs counter-intuitive to the point of being unnecessary. To me, such designs are compelling - it's very plausible that a derived class is stateful, and is not necessarily always in a state where a particular member function can be called. Having the ability to write preconditions that check that the object is in a suitable state seems like a wonderfully useful use case for contracts.

6. Canonical designs

In a so-called canonical design, we want an overrider to have exactly the contract its base has. Now, there's two ways to skin this cat:

1. If you have no need for writing a contract on an overrider, just don't. A call via a base pointer/reference will check the contract on it, and if your implementation has the right stuff, that's all you need.
2. However, for at least two reasons, if you
    - want to make sure that your overrider's contract is checked whenever you compile the definition of the override with enforce/observe semantics, or
    - want to repeat the contract of a base function on an overrider because you want that contract to be eminently visible near the overrider

   you can do so. Just refactor a raw expression in a contract into a predicate function, and call that predicate both in the base function contract and in the overrider contract.

In other words, for the second bullet:

```
 // Example 6.1:
struct Vehicle
{
      virtual void drive(int speed) pre(speed_within_limit(speed));
};

struct WheeledVehicle : Vehicle
{
      void drive(int speed) pre(speed_within_limit(speed) && tires_inflated()) override;
};

struct MotorVehicle : WheeledVehicle
{
      void drive(int speed) pre(speed_within_limit(speed) && tires_inflated() && engine_is_running()) override;
};

void use1(Vehicle* veh)
{
      veh->drive(80);
}

void use2()
{
      MotorVehicle mv;
      use1(&mv);
}
```

Here, for every way of calling a MotorVehicle::drive, it has the same contracts as calling it via a Vehicle*/Vehicle& or a WheeledVehicle*/WheeledVehicle&.

Yes, that requires manual orchestration. But it's doable. It can be made simpler by refactoring the contract of WheeledVehicle::drive() into a function, and reusing that in MotorVehicle. If we want to make it simpler still, we can entertain post-C++26 language extensions that allow saying "give me the same contract as the base function has". The important bit here is that it's possible to express that, and expressing it doesn't require compromising on any of the other design goals this approach has, namely the one mentioned before this one, the ability to narrow/widen a contract of an overrider.

How important would it be to enforce canonical designs?

There's been a lot of debate on this, on the reflectors and elsewhere. I just wish to point out, without all that much elaboration, that a narrowed/narrower contract on an overrider seems perfectly reasonable. If a derived type requires additional setup before it's substitutable for its base, that is explicable and plausible and reasonable. You can't always establish all possible state in a constructor, so it's plausible that

sometimes there's room for a bug where an object of a derived type is passed to code that expects a pointer/reference to a base, and users expect a certain contract on it, and the object of a derived type doesn't yet meet those expectations.

That's fine. Even in all the papers about substitutability, none of them say that substitutability is a purely static concept, and can't have dynamic aspects. A type can be non-substitutable right after construction, and substitutable once certain additional methods are performed on it. Most importantly, contracts can *check* that, once we indeed allow an overrider to have a narrower contract than its base function. That seems incredibly useful.

Thus, summa summarum, considering all the design goals enumerated in this paper, it's more important to allow meeting all those goals than to enforce a particular one at the cost of others. If a user wishes to perform such enforcement, analysis tools and coding guidelines are a plausible way to get it.

7. Multiple inheritance

Multiple inheritance will Just Work, without any additional rules.

If we look at an example like

```
 // Example 7.1:
struct B1 {
    virtual void f(int x) pre(x >= 0);
};

struct B2 {
    virtual void f(int x) pre(x >= 0 && x < 140);
};

struct D : B1, B2 {
    void f(int x) pre(x > 42 && x < 100);
};

void use1(B1* b) {
    b->f(66);
}

void use2(B2* b) {
    b->f(66);
}

void use3()
{
    D d;
    use1(&d);
    use2(&d);
}
```

the example is well-formed and contains no contract violations. The call in use1() will check the contracts of B1::f() and D::f(), the call in use2() will check the contracts of B2::f() and D::f().

Virtual bases and pure virtual functions will also work without any additional special rules:

```
 // Example 7.2:
struct B {
    virtual void f(int x) pre(x >= 0);
};

struct B1 : virtual B {
    virtual void f(int x) pre(x >= 0) = 0;
};

struct B2 : virtual B {
    virtual void f(int x) pre(x >= 0 && x < 140) = 0;
};

struct D : B1, B2 {
    void f(int x) pre(x > 42 && x < 100);
};

void use1(B* b) {
    b->f(66);
}

void use2(B* b) {
    b->f(66);
}
```

```
void use3()
{
    D d;
    use1(&d);
    use2(&d);
}
```

8. Recap

The downsides

This approach

- doesn't provide any language-level mechanisms to check a canonical design to be canonical.
- thus, if such designs are desired to be verified, some sort of other analysis is necessary, has false positives, and may not be applicable at a wide scale due to those false positives.
- doesn't make it very easy to have both the interface and implementation contracts of a virtual call verified.

Rebuttals of the downsides, and the upsides

The rebuttals:

- we can have potentially have language-level mechanisms to check a canonical design to be canonical, post-C++26.
- writing canonical designs isn't altogether hard: use a predicate function in a base function contract, use the same predicate in the overrider's contract.
- it's unlike to be hard to have both the interface and implementation contracts of a virtual call verified, compile both the library and its client with checks turned on.

Further upsides:

- the model is simple; it's not trivially easy to understand if you have presumptions about canonical designs on your mind, but it's easy to explain that the contracts of overrides are independent from the base contracts.
- the rules of the model are simple, the checking of contracts on overrides requires no special rules, multiple inheritance requires no special rules, diamond inheritance with a virtual base requires no special rules.
- the model is general and flexible, and closes no extension doors. It supports all the various use cases brought forward, of which the ability to narrow a contract on an overrider will be very important for stateful derived classes.

Finally, overall, even if the model isn't the most trivially easy to understand, it checks all the design goal boxes thrown at it, and all that is much better than having a contracts ability in C++ that doesn't support one of the fundamentally important parts of C++. My take on this continues to be that we should adopt this approach into the Contracts MVP before it's forwarded for design review by EWG and LEWG.

9. Wording

The wording proposed is as a delta against P2900.

In [dcl.contract.func], remove the restriction that virtual functions can't have contracts:

```
A coroutine ([dcl.fct.def.coroutine]),
~~a virtual function ([class.virtual]),~~ a deleted function ([dcl.fct.def.delete]),
or a function defaulted on its first declaration ([dcl.fct.def.default])
may not have a function-contract-specifier-seq.
```

Modify the modification to [expr.call], paragraph 6

```
When a function is called, each parameter ([dcl.fct])
is initialized ([dcl.init], [class.copy.ctor]) with its corresponding
argument and each precondition assertion([dcl.contract.func)] is evaluated.
If the selected function is virtual, the precondition assertions
of both the statically chosen function and the final overrider are evaluated.
```

Modify the modification to [expr.call], paragraph 7

```
The postfix-expression is sequenced before each expression in the expression-list
and any default argument. The initialization of a parameter,
including every associated value
computation and side effect, is indeterminately sequenced
with respect to that of any other parameter.
These evaluations are sequenced before the evaluation of the precondition
assertions of the statically chosen function,
```

~~which are evaluated in sequence~~
which are, in turn, sequenced before the evaluation of the
precondition assertions of the final overrider, if any.
All precondition assertions of a function are evaluated in sequence ([dcl.contract.func]).

Add a new modification to [expr.call], paragraph 8

```
The result of a function call is the result of the possibly-converted operand
of the return statement (8.7.4) that transferred control
out of the called function (if any), except in a virtual function call
if the return type of the final overrider is different from the return type
of the statically chosen function, the value returned
from the final overrider is converted to the return type of the
statically chosen function.
Then, in a virtual function call, the postconditions of the
statically chosen function are evaluated in sequence ([dcl.contract.func]).
```

10. Some Q&A

Q1:

The proposed wording mentions postconditions, but the design discussion does not. There should be design discussion about why the proposed wording says what it does about postconditions, and why covariance is not considered important.

A1:

There's been an attempt to explain this in the current revision of this paper. But in addition to that, there are plausibly useful designs where the suggested covariance would seem very limiting. We have plausible use cases for both narrowing and widening preconditions, presumably we have such use cases for narrowing and widening postconditions too.

Consider a modified ostream/ofstream pair, a design where output to the stream isn't just silently ignored if the stream can't put it somewhere real, but you don't want a hierarchy-cross-cutting virtual call for verifying it, because you don't need it. You have contracts, you don't need defined-behavior extra APIs to verify things that contracts can check without any overhead in 'ignore' 'mode'. Such an ofstream converts to an ostream, but it has an additional precondition on its output operations that 'is_open()' is true. That's a final overrider precondition that is narrower than that of its base function's precondition. It also has a narrower postcondition, because it's going to actually check that the output request was conveyed into the buffer, and into an actual file buffer, not just a generic streambuf.

If we require/mandate/enfore covariance everywhere, you can't express that kind of designs. But what's even worse, even if you could design differently, you can't just simply check a postcondition that might not be covariant, you would need to revamp your class design.

So yes, this provision is indeed something that supports "non-canonical" designs. For the case of actually *widening* a postcondition, an overrider can act as a new base function, establishing a different contract for itself and its overriders, including a contract where postconditions simply establish less, they establish fewer things. But while such designs were sometimes necessary for practical reasons, they were indeed "type-unsafe" - but now, with contracts, they no longer are, or they are less unsafe. Because with the proposed approach, they can be checked and bug-mitigated!

Q2:

The paper does not show examples with base class state that is visible/used in derived classes (directly or via functions). So to ask for one such example: In the first "Vehicle" example, how would the example change if the Vehicle base class stored its current speed as a data member... could the base class author still use contracts to maintain a meaningful invariant for all Vehicle objects?

A2:

The paper is kinda hinting at it (see Example 6.1), but let's be clearer and more explicit about it:

```
 // Example 10.1:
class Vehicle
{
private:
      int current_speed;
public:
      bool speed_within_limit(int speed) const;
      virtual void drive(int speed) pre(speed_within_limit(speed)); // 10.1.1
};

struct MotorVehicle : Vehicle
{
      bool engineRunning = false;
      void drive(int speed) pre(engineRunning && Vehicle::speed_within_limit(speed)) override; // 10.1.2
};

void use1(Vehicle* veh)
```

```
{
      veh->drive(80); // 10.1.3
}

void use2()
{
      MotorVehicle mv;
      use1(&mv);
      mv.drive(400);
}
```

The proposal doesn't provide a particular facility for the base class to enforce that derived classes use the same predicate as the base does. But there are techniques like the one above that make it relatively easy to do. Actual enforcing mechanisms are left for future extensions, although there's no particular guarantee that we'll get such extensions. But nevertheless, the call at 10.1.3 evaluates both the precondition at 10.1.1 and the precondition at 10.1.2, because the one at 10.1.1 is evaluated because we called through a Vehicle, so that's the "entry point"/interface/handle contract check that gets evaluated, and the final overrider is 10.1.2, so the contract check of that gets evaluated as well.

Q3:

More generally: The paper does not mention invariants, and even though the MVP doesn't have invariants, future extensibility is a reasonable question and invariants are important even today... Would the paper also argue that derived class invariants should by default be independent of base class invariants?

A3:

Maybe. That depends on how those invariants would work. It's also worth remembering that base class invariants would need to be accessible to derived classes for them to be able to check the base invariants. This proposal doesn't propose any special access rules via which preconditions and postconditions of bases could just always be checked, and maybe we shouldn't do that for invariants either. There's also the case of private bases, where there's ostensibly no need for a derived class to redo its invariant checks.

Q4:

This feels like the rules for template specialization / duck typing (you get the semantics/contracts of the function you happen to match + specializations are not substitutable and need not bear any relationship to the primary template at all), which have generally been viewed as a language weakness that we've been trying to correct (e.g., with concepts, and hang-wringing about vector<bool>). What reasons are there to pursue a design for inheritance that's more like that, than like existing language rules for inheritance (e.g., covariance)?

Q5:

The paper acknowledges that substitutability is desirable, but argues that contracts should be non-substitutable by default and that manual orchestration should be required to get substitutability. In an era where we are trying to increase C++'s type safety, and reduce type safety bugs rather than create new ways to write them, how would we answer a question like "when C++ finally added contracts why did it make the new feature type-unsafe again"?

The paper argues that substitutability is desirable, *in some cases*. There are field experience reports of existing code where strict rules of that aren't always followed. We could do a substitutability-enforcing design first, and a relaxation later, but that has two problems:

- Substitutability doesn't mean that an object is always substitutable after its construction. Nothing in e.g. Liskov's paper says that. It's perfectly plausible that there are types where substitutability changes over time. Overriding functions that have narrower preconditions than their base are wonderful; they can check perfectly reasonable designs where an object needs post-construction operations to become *actually* substitutable for its base type. There's *nothing* wrong with such designs, it's perfectly reasonable to create a bag of objects in a container without necessarily incurring the overhead of getting them into such a substitutable state. You don't start the engines of all of your motor vechicles the moment the vehicles roll out from the assembly line and into a car dealer's storage.
- This approach, whether we all agree on it or not, allows contractifying the "non-canonical" designs *now* instead of in an unknown point in the future. You can start slapping contracts on base class functions - there's no need to say "don't do that unless your derived classes have a canonical design". Just go ahead and start using contracts, the design comes with escape hatches. It comes with escape hatches by default. :) For some code that we have field reports of, it makes it much easier and faster to migrate to contract use.

Q6:

Could the paper elaborate more on why the wording says to only fire the pre/post conditions of the static and most-derived types? For example, for a linear hierarchy from most-base A to most-derived E with a virtual function A::f, is it intended that a call with static type B::f that dynamically calls E::f will fire the pre/post conditions for B::f and E::f, but not for A::f, C::f, or D::f?

A6:

An attempt to elaborate on that point has been made in 2. Proposed semantics.

Q7:

The paper has no references/citations: What experience with prior art exists for the proposed semantics?

A7:

The most important design rationale for these semantics wasn't trying to be like another language, and no library approach can truly achieve what preconditions and postconditions do. I'm not sufficiently familiar with e.g. BSL's assertion mechanisms to be able to say to what extent and how closely they are able to mimic the language facility. The design goals were a significant driving factor, the goal of having no ABI impact was deemed essential. Various other parts of the proposal were designed with WG21 members' field experience reports in mind, especially ones about both narrowing and widening preconditions.

Curiously enough, this document about ADA suggests that ADA has both preconditions and postconditions that are inherited (the "class wide" ones) and preconditions and postconditions that are *not* inherited. In that description, the "class wide" contracts are inherited, and their full chain "must be true", but it also says thus:

> However, the rules regarding preconditions are perhaps surprising. The specific precondition Pre for Equilateral_Triangle must be true (checked in the body) but so long as just one of the class wide preconditions Pre'Class for Object and Triangle is true then all is well. Note that class wide preconditions are checked at the point of call. Do not get confused over the use of the word apply. They all apply but only the ones seen at the point of call are actually checked.

The non-"class wide" contracts are not inherited, and don't run as a chain.

There's also nothing in that writeup that suggests any substitutability is enforced. There's a "must be true" statement about a "class-wide" chain, but nothing in any of it says that e.g. a precondition on an override couldn't be narrower than the one of its base.

There's certainly a difference that the contracts described in that writeup are always attached to a "subprogram"/procedure body, so abstract procedures can't have contracts. That's certainly another difference between ours and theirs, there's no restriction that a pure virtual function can't have contracts - and of course there is a more fundamental difference that in C++, a pure virtual function can have a definition.

11. A look at some suggested requirements

These were contributed by another WG21 expert member. Some of them are certainly contradictory.

1. Allow widening preconditions in derived classes
   - This proposal allows that.
2. Allow narrowing preconditions in derived classes
   - This proposal allows that.
3. Ensure substitutability (preconditions cannot be narrowed and postconditions cannot be widened when calling through reference to base)
   - This proposal doesn't include mechanism for such ensuring.
4. Don't silently inherit contracts (check only the derived function contract when making a non-virtual function call)
   - This proposal does that.
5. Allow checking both the interface and the implementation contracts when making a virtual function call
   - This proposal does that.
6. Allow checking contracts through the entire hierarchy of classes when making a virtual function call
   - This proposal does do a full-chain check.
7. Do not check contracts that are not relevant for the correctness of the given virtual function call
   - This proposal doesn't check non-relevant contracts, for designs where some contracts in an inheritance hierarchy aren't deemed relevant.
8. Avoid an ABI break when adding pre/post to a virtual function
   - This proposal does avoid such ABI breaks, and that's deemed fundamentally important.
9. Do not require client-side checking
   - This proposal doesn't require client-side checking, but it's certainly the most plausible high-QoI implementation strategy for checking a base function contract for a virtual call.
10. Do not require recompilation when a contract changes
    - This proposal doesn't require recompiling a base class when a contract on a derived class changes. Recompiling a derived class when a base contract changes isn't required either.
11. Support multiple inheritance where the base class functions have different contracts
    - This proposal supports multiple inheritance with base class functions from different base classes having different contracts.
12. Have implementation experience with the chosen strategy
    - We unfortunately do not have implementation experience for this strategy yet.
13. Interfaces and implementations should be independent
    - This proposal allows base interfaces and overrider implementations to be independent.
14. Avoid adding more symbols to the binary when adding a contract to a virtual function
    - At various times, implementation strategies that rely on additional functions, exported from library objects, to be used for e.g. checking a base class contract on a virtual call. This proposal doesn't require that.